

Lecture-2

From Render Tree to Virtual DOM

How the browser builds a page — and how React re-imagines it



DOM &
CSSOM



Render Tree



Virtual DOM



Diffing &
Reconciliation

Quick recap, then today's roadmap

Last time: the browser's 3 engines — HTML parser, CSS parser, JS engine



DOM

HTML parser builds the structure tree



CSSOM

CSS parser builds the style tree



Render Tree

Browser merges DOM + CSSOM



ReactJS

A library that manages UI updates



Virtual DOM

React's in-memory copy of the UI

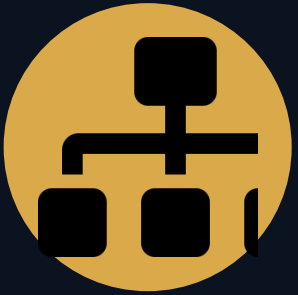


Diff + Reconcile

React finds & applies the changes

DOM — Document Object Model

Built by the browser's HTML parser as it reads your markup top to bottom



A tree of objects, one per HTML tag

`<html> → <body> → <div> → <p> ...` each becomes a node the browser can read, style, and manipulate.

It only knows structure — not how anything looks yet.

```
html
├─ body
│   ├── header
│   └── div.app ← the single root node React will take over
```

CSSOM — CSS Object Model

Built by the browser's CSS parser from stylesheets, <style> tags, and inline styles



A tree of computed style rules

For every DOM node, the CSSOM works out the final color, size, margin, and position — after cascading and inheritance.

DOM = what exists. CSSOM = how it should look.

```
body { font-size: 16px }  
  div.app { color: inherit; padding: 20px } ← computed per node, not per rule
```

Render Tree = DOM + CSSOM

The browser combines both trees into the exact pixels it will paint



Only visible nodes make it in

`display:none` elements are skipped entirely — they exist in the DOM but never reach the render tree.

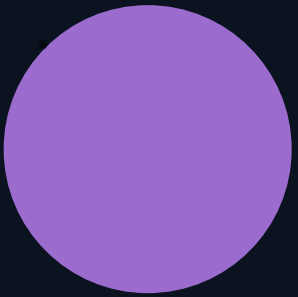
Layout → Paint → Composite

Layout: compute exact size/position of every box. Paint: fill in pixels — text, colors, borders. Composite: layers combined onto the screen.

This whole DOM → CSSOM → Render Tree → Layout → Paint pipeline is exactly what a real DOM update has to repeat.

Enter ReactJS

A JavaScript library for building UI — not a browser engine, a tool that sits on top of the DOM



The problem it solves

Directly touching the real DOM (adding/removing nodes) forces the browser to redo layout & paint — expensive, especially for frequent UI updates.

React's idea: never touch the real DOM directly. Describe the UI in JS, let React figure out the minimal real change.

Components

JSX

State & Props

Virtual DOM

Virtual DOM — what it is, where it lives

A plain JavaScript object tree that mirrors the UI — created and kept entirely in the browser



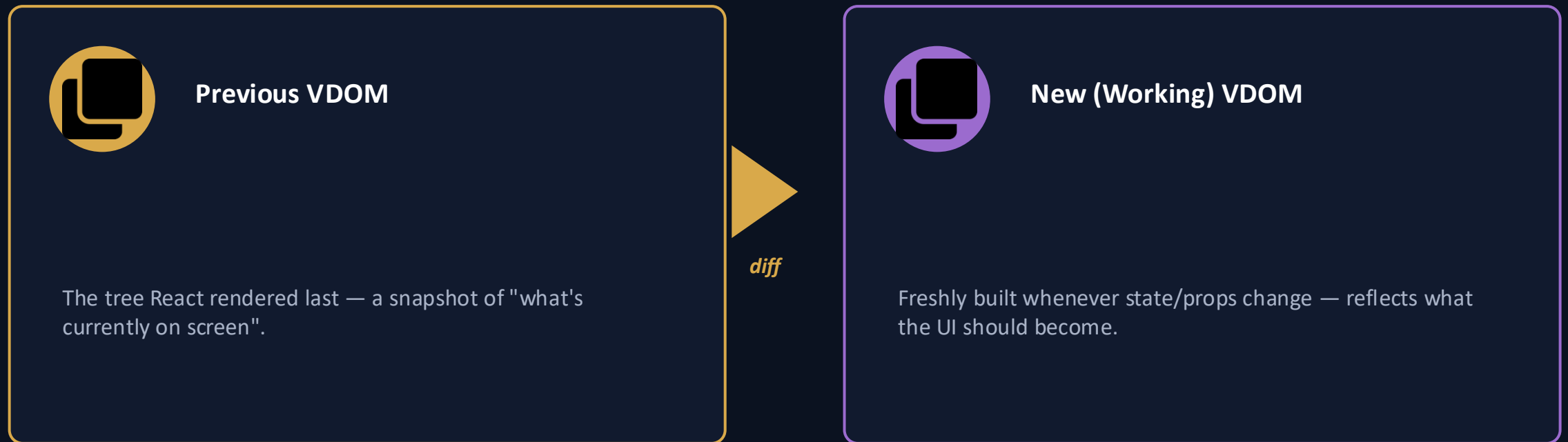
Created by: the React library itself — specifically `react` (builds the JS-object tree from your JSX/components) + `react-dom` (knows how to translate it into real DOM operations).

Created where: in browser memory (client-side JS), never on the server. The server only ever ships the static JS bundle — React executes and builds the Virtual DOM after that bundle runs in the user's browser.

```
{ type: 'div', props: { className: 'app' },  
  children: [ { type: 'h1', props: {}, children: ['Hello'] } ] }
```

How many Virtual DOMs exist?

React always keeps two in-memory trees around, per component tree, at any moment of an update



Diffing algorithm

React compares the previous and new VDOM tree node-by-node, using fast heuristics — not a generic tree diff



Different element type → replace subtree

If `<div>` becomes `<span>`, React tears down the old node and builds fresh — no attempt to reuse.

Same type → update attributes in place

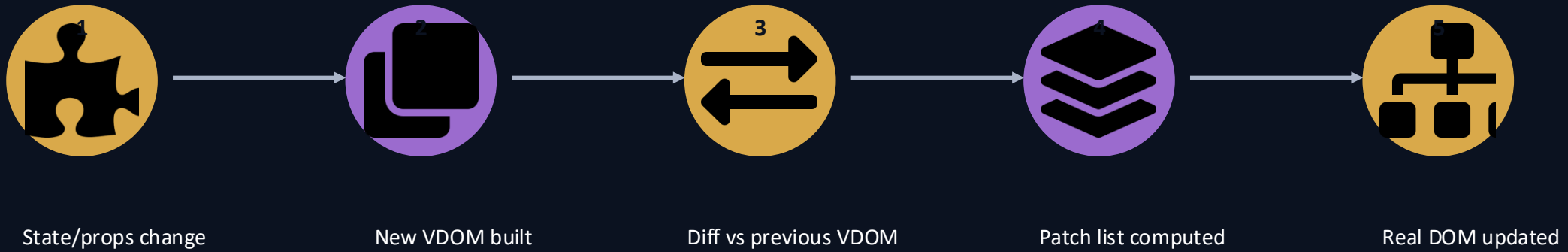
If `<div>` stays `<div>`, React keeps the node and only patches changed props/attributes.

Lists need a key

Without a stable key, React compares by position and may re-render everything; a key lets it match items across re-orders.

Reconciliation

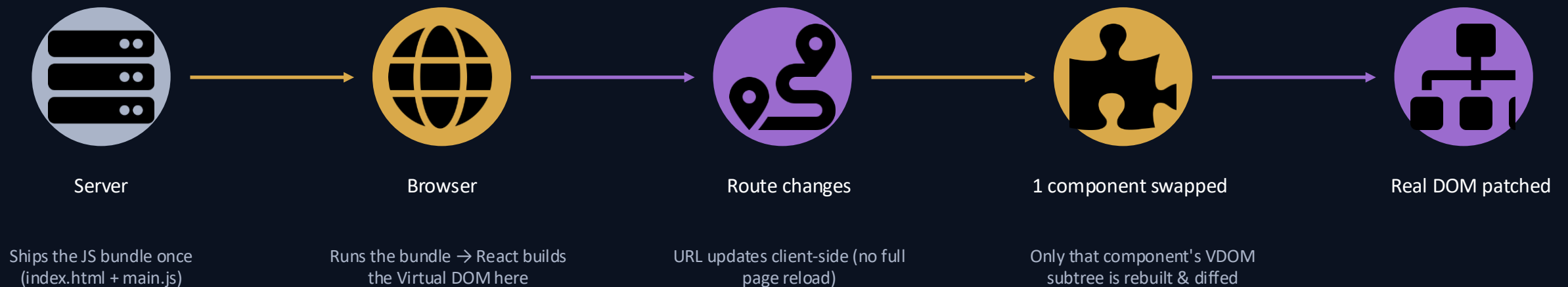
The overall process: diff the trees, compute the minimal patch, then apply it to the real DOM



Only the actually-changed DOM nodes are touched — React never re-triggers the whole DOM → CSSOM → Render Tree pipeline for the whole page, only for the patched nodes. This is exactly why VDOM-based updates outperform naive full-page real-DOM rewrites.

SPA route change: where does the VDOM actually sit?

React app is hosted on a server, but the Virtual DOM is built and diffed entirely inside the user's browser



A React SPA is built and hosted as static files. Once loaded, the whole DOM/CSSOM/Render Tree/VDOM cycle for navigation happens client-side in the browser's memory — the server is only consulted again for API data, not for building or diffing the VDOM.

Loading scripts: `async` vs `defer` vs plain `<script>`

How each one affects HTML parsing, execution timing, and order

Feature	<code>async</code>	<code>defer</code>	<code><script></code>
HTML Parsing	Parallel download; pauses parsing only during script execution.	Parallel download; completely unblocked parsing.	Pauses parsing entirely for download and execution.
Execution Timing	Immediately after downloading finishes.	Only after HTML parsing is completely finished.	Instantly upon being discovered in the DOM.
Execution Order	Independent (first downloaded, first executed).	Guaranteed (follows document order).	Strictly in document order.
DOM Dependent	No, can execute before the DOM is fully built.	Yes, executes right before <code>DOMContentLoaded</code> fires.	Yes or no, depending on the script's location.

Thank You !